

Prompt

Teach me SCP for AWS EC2 and Linux administration.

Cover:

- * Why SCP exists
- * How SCP uses SSH
- * Production use cases

Then teach:

1. Local → EC2
2. EC2 → Local
3. Directory transfer
4. Server → Server transfer (first we have to do passwordless authentication)

For each:

- * Command breakdown
- * Practical implementation
- * Verification

Finally:

- * SCP vs RSYNC
- * Advantages & limitations
- * 10 interview questions

Keep it concise, practical, and interview-focused.
Wait for my confirmation before moving ahead.

SCP

SCP (Secure Copy Protocol) — AWS EC2 & Linux Administration

We'll do this in parts and stop after the foundation. Once you're comfortable, we'll move to **Local ↔ EC2 transfers**.

1. Why SCP Exists

Before SCP, administrators commonly used:

- FTP
- RCP (Remote Copy)

Problem:

- Data transferred in plain text
- Credentials exposed
- No encryption

SCP was introduced to provide:

- ✓ Secure file transfer
- ✓ Authentication via SSH
- ✓ Encrypted data in transit
- ✓ Simple command-line usage

Think of SCP as: "cp command for remote systems."

Local copy:

```
cp file.txt /tmp/
```

Remote copy:

```
scp file.txt user@server:/tmp/
```

2. How SCP Uses SSH

This is the most important interview concept.

SCP does **not** create its own secure channel.

Instead:

SCP

|



SSH Connection

|



Encrypted Transfer

When you run:

```
scp -i mykey.pem file.txt ec2-user@54.x.x.x:/home/ec2-user/
```

SCP:

Step 1

Uses SSH to authenticate.

-i mykey.pem

Private key proves your identity.

Step 2

SSH session established.

Your Laptop

|

SSH

|

EC2

Step 3

File transferred through encrypted SSH tunnel.

No separate port required.

Network Requirement

Only:

TCP 22

must be accessible.

Security Group:

Inbound:

SSH (22)

Source: Your IP

No SCP-specific port exists.

Interview question:

Which port does SCP use?

Answer:

Port 22 (SSH)

SCP Authentication Methods

Method 1: Password Authentication

Linux server:

```
scp file.txt user@server:/tmp/
```

Server asks:

password:

Common on on-prem servers.

Method 2: Key-Based Authentication

AWS standard approach.

```
scp -i mykey.pem file.txt ec2-user@server:/tmp/
```

Uses:

Private Key

+

authorized_keys

No password required.

Production Use Cases

1. Deploy Application Files

Upload application package:

Laptop

|

SCP

▼

EC2

Example:

app.zip

config.yaml

nginx.conf

2. Backup Retrieval

Download logs:

EC2

|

▼

Laptop

Example:

/var/log/messages

/application/logs

3. Configuration Migration

Move configs between servers:

Server A

|

SCP



Server B

Examples:

nginx.conf

httpd.conf

systemd files

4. SSL Certificate Transfer

Transfer:

.crt

.key

.pem

between administration servers.

5. Database Export Movement

Move:

mysqldump.sql

backup.tar.gz

to backup servers.

Key Points

- SCP depends on SSH.
 - Port 22 must be reachable.
 - Same key pair used for SSH can be used for SCP.
 - Security Group must allow SSH access.
-

Interview Nuggets

Difference between SSH and SCP

SSH:

Remote login

SCP:

File transfer using SSH

Can SCP work without SSH?

No.

Is SCP encrypted?

Yes.

Entire transfer is encrypted by SSH.

Does SCP preserve permissions?

Basic file permissions are copied.

For preserving timestamps and metadata:

```
scp -p file.txt server:/tmp/
```

What you've learned so far

- ✓ Why SCP exists
 - ✓ How SCP works internally
 - ✓ SCP + SSH relationship
 - ✓ Authentication methods
 - ✓ Production use cases
 - ✓ AWS networking requirement (Port 22)
-

Scenario 1: Local → EC2

Command

```
scp -i mykey.pem file.txt ec2-user@54.x.x.x:/home/ec2-user/
```

Command Breakdown

Part	Meaning
scp	Secure Copy
-i mykey.pem	SSH private key
file.txt	Source file
ec2-user	Remote user
54.x.x.x	Public IP/DNS
/home/ec2-user/	Destination

Practical Example

Local:

ls

Output:

file.txt

Upload: scp -i mykey.pem file.txt ec2-user@54.x.x.x:/home/ec2-user/

Verification

Download from server.

Command

```
scp -i mykey.pem ec2-user@54.x.x.x:/var/log/messages .
```

Breakdown

Part	Meaning
scp	Secure Copy
-i mykey.pem	SSH Key
ec2-user@54.x.x.x	Remote system
/var/log/messages	Source file
.	Current local directory

Practical Example

Download:

```
scp -i mykey.pem ec2-user@54.x.x.x:/etc/os-release .
```


Verification

Local:

```
cat os-release
```

File should exist locally.

Scenario 3: Directory Transfer

Without recursive option:

```
scp mydir ec2-user@server:/tmp/
```

Fails.

Because SCP copies files by default.

Recursive Copy

```
scp -r -i mykey.pem mydir ec2-user@54.x.x.x:/home/ec2-user/
```

[illegible]

Example

Local:

project/

├─ app.py

├─ config.yml

└─ logs/

Transfer:

```
scp -r -i mykey.pem project ec2-user@54.x.x.x:/home/ec2-user/
```

Verification

EC2:

```
ls -R project
```

Entire directory should appear.

```
[ec2-user@ip-172-31-43-112 ~]$ ls -R kkkkk
kkkkk:
AditiNarangPhoto.jpeg  abcd.pdf  docker.pdf  file1  file2  file3
[ec2-user@ip-172-31-43-112 ~]$ |
```

Useful SCP Options

Preserve Timestamp & Permissions

```
scp -p file.txt ec2-user@server:/tmp/
```

Verbose Mode

Troubleshooting:

```
scp -v file.txt ec2-user@server:/tmp/
```

Shows SSH negotiation details.

Limit Bandwidth

```
scp -l 5000 file.txt ec2-user@server:/tmp/
```

Useful in production.

Scenario 4: Server → Server Transfer

This is a favorite interview topic.

Why Passwordless Authentication First?

Suppose:

Server A

Server B

A must authenticate to B.

If no passwordless setup exists:

```
scp file.txt ec2-user@serverB:/tmp/
```

will ask:

password:

Production automation cannot rely on manual password entry.

Therefore:

SSH Key-Based Authentication (This is done on Day 1, Following step1,2,3 are from day1)

is configured first.

Step 1: Generate Key on Server A

Login to Server A:

ssh-keygen

Files created:

~/.ssh/id_rsa

~/.ssh/id_rsa.pub

Step 2: Copy Public Key to Server B

View public key: cat ~/.ssh/id_rsa.pub

Append contents to: ~/.ssh/authorized_keys

on Server B.

Step 3: Verify Passwordless Login

From Server A:

ssh ec2-user@serverB

Expected:

Login succeeds

No password prompt

Step 4: SCP Server → Server

Now copy:

```
scp file2 file3 ec2-user@<server-2-ki-ip>:/home/ec2-user/data/
```

Server1:

```
[ec2-user@ip-172-31-43-112 ~]$ ssh-keygen
Generating public/private rsa key pair.
Enter file in which to save the key (/home/ec2-user/.ssh/id_rsa):
/home/ec2-user/.ssh/id_rsa already exists.
Overwrite (y/n)?
[ec2-user@ip-172-31-43-112 ~]$
[ec2-user@ip-172-31-43-112 ~]$ n
-bash: n: command not found
[ec2-user@ip-172-31-43-112 ~]$ ls
[ec2-user@ip-172-31-43-112 ~]$ ls -a
. . . bash_history .bash_logout .bash_profile .bashrc .ssh
[ec2-user@ip-172-31-43-112 ~]$ cd .ssh
[ec2-user@ip-172-31-43-112 .ssh]$ ls
authorized_keys id_rsa id_rsa.pub
[ec2-user@ip-172-31-43-112 .ssh]$ cat id_rsa.pub
ssh-rsa AAAAB3NzaC1yc2EAAAADAQABAAQGDjsgpdrR3yIfoSr8LFbOGmqPZvSwKoQ18D4ujYJ11L4JwANbCyebZ25caL+7CEbEGSEU7VS8fWuRdIMyW0fBnG06XG590rKI6MOV2Y8MhFfc59QneG3
nktW+oHd1rlFz/VphWz95HW6beBsutGYT6ji0VxiJw1TDaezHTGKkHEVIEcIicPTNuFCL+VK+kHweJEY7iLcA3z5/hdBR4kCzKF5LnQMhCCjpr5Yw89RNDgkXyIXG4DK6/M106DFWzUDmXWw0qbkzt8NdLM
JXGAwAZLm6otlrW8lp4rormGIIP5WE3nTWAsIMCDaNDZ3cMSwwAf71g80bDZtxe0DChBKvs6npRASvrJhjs2QNAb+Arx4R0TZTMoyJyMjH3aZrTSEfuUEFTqdQ//ptHUI0HsURDsX9GPhrYSc4Wb2FL5uFmh
YhM1HJnfc7F/sUZb9WAgILtjotThi13JcdnkjPxfVBexSVtAXjfOK9c9rYa0/aAmqjwTJei8ulJyZ5hNwE= ec2-user@ip-172-31-43-112.ap-south-1.compute.internal
[ec2-user@ip-172-31-43-112 ~]$ cd ..
[ec2-user@ip-172-31-43-112 ~]$ ls
[ec2-user@ip-172-31-43-112 ~]$ touch file1 file2 file3
[ec2-user@ip-172-31-43-112 ~]$ echo "Hi Everyone" >> file1
[ec2-user@ip-172-31-43-112 ~]$ echo "Aditi Narang here" >> file2
[ec2-user@ip-172-31-43-112 ~]$ echo "Hope you are well" >> file3
[ec2-user@ip-172-31-43-112 ~]$ scp file1 ec2-user@43.205.120.133
[ec2-user@ip-172-31-43-112 ~]$ scp file1 ec2-user@43.205.120.133:/home/ec2-user/data/
The authenticity of host '43.205.120.133 (43.205.120.133)' can't be established.
ED25519 key fingerprint is SHA256:vVT58ECIs2+ctJbFG/otAe4gNwWBy7t3eu8L4nBW6H.
This key is not known by any other names
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '43.205.120.133' (ED25519) to the list of known hosts.
file1 100% 12 35.2KB/s 00:00
[ec2-user@ip-172-31-43-112 ~]$ scp file2 file3 ec2-user@43.205.120.133:/home/ec2-user/data/
file2 100% 18 48.5KB/s 00:00
file3 100% 18 55.7KB/s 00:00
[ec2-user@ip-172-31-43-112 ~]$
```

Verification

On Server B:

cd data

ls data

Server2:

```
[ec2-user@ip-172-31-43-112:~] ssh ec2-user@ip-172-31-39-18 -t + ~
```

```
Warning: Permanently added '43.205.120.133' (ED25519) to the list of known hosts.
```

```
#  
##### Amazon Linux 2023  
###  
#####  
#####  
#####  
#/  
V--> https://aws.amazon.com/linux/amazon-linux-2023  
  
#####  
#####  
#####  
m/
```

```
[ec2-user@ip-172-31-39-18 ~]$ cd .ssh  
[ec2-user@ip-172-31-39-18 .ssh]$ ls  
authorized_keys  
[ec2-user@ip-172-31-39-18 .ssh]$ cat authorized_keys  
ssh-rsa AAAAB3NzaC1yc2EAAAADAQABAAQDLBfiDoy6twzz5Wbv7q3hbwo9HjJInbXu35Gfcs/PDHrb1kP045zWBDnIXrW3ILelY0Z0uHSAGlWDw9o9WS8NuT7HY5LBIQ0FOCV+OpCR4v1OSX96R5Y  
HP9NGKxJlydhdlIvd3pp72wUbarneJlmQyrlapf+siVT-zUSCpjbbSeNBIOIjmhk615Tp+kCuI8PCuHQFTHGICgoioXpyrz2LFh2qO29os7Zubxm92swQCjgmMoteKYRzdPrJop8ORQLuu52NvTLFLXLtXQV4  
4LC4nggywm6BVfBGdGUJJllakaHA78icOrkzGYLoF/p1OLSgYVqmWn5tfXkaA0Q2ye7plvh narang  
[ec2-user@ip-172-31-39-18 .ssh]$ vim aut  
[ec2-user@ip-172-31-39-18 .ssh]$ vim aut  
aut authorized_keys  
[ec2-user@ip-172-31-39-18 .ssh]$ vim authorized_keys  
[ec2-user@ip-172-31-39-18 .ssh]$ cd ..  
[ec2-user@ip-172-31-39-18 ~]$ ls  
[ec2-user@ip-172-31-39-18 ~]$ mkdir data  
[ec2-user@ip-172-31-39-18 ~]$ ls  
data  
[ec2-user@ip-172-31-39-18 ~]$ whoami  
ec2-user  
[ec2-user@ip-172-31-39-18 ~]$ ls  
data  
[ec2-user@ip-172-31-39-18 ~]$ ls  
data  
[ec2-user@ip-172-31-39-18 ~]$ ls  
data  
[ec2-user@ip-172-31-39-18 ~]$ cd data  
[ec2-user@ip-172-31-39-18 data]$ ls  
file1  
[ec2-user@ip-172-31-39-18 data]$ ls  
file1 file2 file3  
[ec2-user@ip-172-31-39-18 data]$ |
```

Common Errors

Permission Denied

Permission denied (publickey)

Reasons:

- Wrong key
- Wrong username
- Missing `authorized_keys` entry
- Incorrect permissions

Connection Timed Out

ssh: connect to host ...

port 22: Connection timed out

Usually:

- Security Group issue
- NACL issue
- Route issue
- Instance not running

No Such File

scp: file.txt: No such file

Source path incorrect.

SCP vs RSYNC

Feature	SCP	RSYNC
Encryption	Yes (SSH)	Yes (SSH)
Simple	Yes	Moderate
Resume Transfer	No	Yes
Incremental Copy	No	Yes
Bandwidth Efficient	No	Yes

Large Data Sync	Poor	Excellent
Directory Sync	Basic	Advanced
Production Backup	Rare	Common

When to Use SCP

- ✓ Small file transfers
 - ✓ Quick deployments
 - ✓ Log downloads
 - ✓ Configuration uploads
 - ✓ One-time copy operations
-

When to Use RSYNC

- ✓ Large datasets
 - ✓ Daily backups
 - ✓ Synchronization jobs
 - ✓ Repeated transfers
 - ✓ Production backup systems
-

Advantages of SCP

Secure

Uses SSH encryption.

Easy

Single command.

Available Everywhere

Installed on almost all Linux systems.

No Additional Ports

Uses:

22/TCP

only.

Limitations of SCP

No Resume

If transfer stops at 99%:

Start again from 0%

No Incremental Sync

Copies everything every time.

Slow for Large Repeated Transfers

Compared with RSYNC.

Limited Features

Primarily designed for copying.

Not synchronization.

10 Interview Questions

1. What port does SCP use?

Port 22 (SSH).

2. Does SCP have its own authentication mechanism?

No. Uses SSH authentication.

3. Can SCP work without SSH?

No.

4. Difference between SSH and SCP?

SSH = Remote login

SCP = File transfer over SSH

5. How do you copy a directory using SCP?

```
scp -r directory user@server:/path
```

6. How do you preserve timestamps?

```
scp -p file.txt user@server:/path
```

7. Why is SCP not ideal for backups?

No resume and no incremental transfer.

8. SCP vs RSYNC?

RSYNC transfers only changed blocks and can resume interrupted transfers.

9. What causes **Permission denied (publickey)**?

Wrong key, username, permissions, or missing public key.

10. How do two EC2 instances perform SCP without passwords?

Configure SSH key-based authentication:

Server A public key

↓

Server B authorized_keys

Then SCP works non-interactively.

Interview Summary (Must Remember)

SCP = Secure Copy Protocol

Uses SSH

Uses Port 22

Local → EC2

EC2 → Local

Directory Copy (-r)

Server → Server :

 Passwordless authentication required for automation

 Good for:

- Small transfers
- Quick deployments

RSYNC preferred for:

- Backups
- Large data
- Synchronization